

**Aim:**

Write a python program to implement DFS

**Objectives:**

The objective of the Depth-First Search (DFS) algorithm is to traverse or search all the nodes of a graph or tree by exploring one branch as deeply as possible before backtracking. It starts from a source node, visits each unvisited node, and continues until all reachable nodes are explored. DFS is commonly used for graph traversal, path finding, cycle detection, and solving problems such as mazes and connected components.

**Algorithm:**

1. Start.
2. Create an empty stack and a visited array.
3. Push the starting node onto the stack and mark it as visited.
4. While the stack is not empty:  
Pop the top node from the stack.  
Visit (process) the node.  
Push all unvisited adjacent nodes onto the stack and mark them as visited.
5. Repeat until the stack becomes empty.
6. Stop.

**Flow chart:**

**Source Code:**

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

start = 'A'

visited = set()

def dfs(node):
    if node not in visited:
        visited.add(node)
        print(node, end=" ")
        for neighbor in graph[node]:
            dfs(neighbor)

print("DFS Traversal:")
dfs(start)
```

**Result:**

The Depth-First Search (DFS) algorithm was successfully executed. The graph was traversed by visiting nodes depth-wise, and all reachable nodes were visited exactly once.

**Output:**

```
main.py  Run  Output  Clear
1 graph = {
2     'A': ['B', 'C'],
3     'B': ['D', 'E'],
4     'C': ['F'],
5     'D': [],
6     'E': [],
7     'F': []
8 }
9
10 start = 'A'
11
12 visited = set()
13
14 def dfs(node):
15     if node not in visited:
16         visited.add(node)
17         print(node, end=" ")
18         for neighbor in graph[node]:
19             dfs(neighbor)
20
21 print("DFS Traversal:")
22 dfs(start)
```

DFS Traversal:  
A B D E C F  
== Code Execution Successful ==